

GPU Rental & Research Portal – Project Documentation

Table of Contents

- 1. [Project Overview](#)
- 2. [Key Features](#)
- 3. [System Architecture](#)
- 4. [Technology Stack](#)
- 5. [Authentication & Security](#)
- 6. [AI Assistant Integration](#)
- 7. [GPU Recommendation Logic](#)
- 8. [Database Schema Overview](#)
- 9. [API Endpoints](#)
- 10. [Installation & Setup](#)
- 11. [Usage Guide](#)
- 12. [Future Enhancements](#)
- 13. [Credits](#)

Project Overview

This project is a **GPU Rental and Research Support Platform** built using React.js. It helps users select and rent appropriate GPUs by analyzing their preferences like region, OS, budget, and usage time. The system includes a **smart AI assistant** and **automated recommendation logic** based on real-time inputs.

Key Features

- Smart GPU recommendation logic (based on multiple constraints)
- AI-powered assistant for queries and suggestions
- REST API-based backend
- JWT-authenticated login system
- Admin panel for request management
- Modern UI built in **React.js**
- Demo account for quick testing

System Architecture

Frontend (React.js + Tailwind)

|

|-- REST API Calls

|

Backend (Node.js + Express)

|

|-- Business Logic

|-- AI Assistant Integration

|

Database (MongoDB)

Technology Stack

Backend & Server

Technology	Description
Node.js	JavaScript runtime for backend logic
Express.js	Lightweight backend framework for REST APIs

Frontend & UI

Technology	Description
React.js	Component-based UI framework
Tailwind CSS	Utility-first CSS framework

Authentication & Security

Technology	Description
JWT	Token-based authentication
Middleware	Secures routes and resources

Database

Technology	Description
MongoDB	NoSQL database storing users, GPU data, and requests

AI Assistant

Technology	Description
OpenAI API	For user queries, GPU advice, and research help
LLM	Powers language understanding and natural answers

Authentication & Security

- **JWT** used for login sessions
- **Admin roles** have protected access via middleware
- **Password hashing** for secure credential storage

AI Assistant Integration

- **Purpose:** Helps with GPU selection, answers AI/ML-related questions, and guides users.
- **Context-Aware:** Learns from user input and dynamically responds.
- **Endpoints:** /api/ai-assistant to handle AI queries.

GPU Recommendation Logic

The **recommendation engine** works as follows:

1. **Primary Filter – Region Match:**
Only GPUs available in the selected region are considered.
2. **Secondary Filter – OS Match:**
Match only with user-preferred Operating System (Linux, Windows, etc.).
3. **Budget Constraint:**
GPUs are shortlisted if:

```
(price_per_hour * usage_hours) <= budget
```

1. **Output:**

A sorted list of GPU instances satisfying the conditions is returned, sorted by ascending cost.

2. **Response Fields:**

```
{  
  
  "instance_id": "String",  
  
  "cost": "Number",  
  
  "ram": "String",  
  
  "class": "e.g. NVIDIA A100",  
  
  "hourly_rate": "Number"  
}
```

This result is then displayed in the frontend with clean card layouts.

Database Schema Overview

User Schema

```
{  
  
  "name": "String",  
  
  "email": "String",  
  
  "password": "Hashed String",  
  
  "role": "user | admin"  
}
```

GPU Schema

```
{  
  
  "region": "String",  
  
  "os": "String",  
  
  "class": "String",  
  
  "ram": "String",  
  
  "price_per_hour": "Number"
```

```
}
```

Request Schema

```
{
```

```
  "userId": "ObjectId",
```

```
  "region": "String",
```

```
  "os": "String",
```

```
  "budget": "Number",
```

```
  "usage_hours": "Number",
```

```
  "status": "pending | approved | rejected"
```

```
}
```

API Endpoints

Method	Endpoint	Description
POST	/api/auth/register	Register a new user
POST	/api/auth/login	User login
POST	/api/submit-request	Submit GPU rental request
POST	/api/recommend	Get recommended GPU instances
POST	/api/ai-assistant	Ask question to AI assistant
GET	/api/admin/requests	Admin fetches all requests

Installation & Setup

Prerequisites

- Node.js v16+
- MongoDB URI

- OpenAI API Key

Environment Setup

git clone https://github.com/your-repo/gpu-portal.git

cd gpu-portal

npm install

Create a .env.local file:

ini

CopyEdit

MONGODB_URI=your_mongo_uri

JWT_SECRET=your_jwt_secret

OPENAI_API_KEY=your_openai_key

Run the App

bash

CopyEdit

npm run dev

Demo Account

To test quickly:

- **Email:** demo@gpu.com
- **Password:** Demo@123
- Access Level: Standard User
- Can test submission form and view AI assistant

Usage Guide

1. Register/Login or use the **demo account**
2. Fill GPU rental form
3. View **recommended instances**
4. Chat with **AI assistant** for help
5. Admin can approve/reject requests

Future Enhancements

- Payment gateway integration

- Deployment monitoring (uptime, GPU queue status)
- Real-time chat with support/admin
- User analytics and history

Credits

- [OpenAI](#)
- [React.js](#)
- [Tailwind CSS](#)
- [MongoDB](#)
- [JWT.io](#)